

Sistemi Operativi

Corso di Laurea in Informatica

Sapienza Università di Roma

a.a. 2026/2027

Marco Raoul Marini

marini@di.uniroma1.it

Gabriele Tolomei

tolomei@di.uniroma1.it

Lecture 2.2: Virtualization II

CPU Scheduling: Basic Policies (FIFO, SJF, STCF, RR)

Opening Thought

"How should the OS decide which process to run next? To understand this, we must build a framework of scheduling policies. We start with simple policies, analyze their limitations under realistic workloads, and iteratively design better algorithms."

— Remzi & Andrea Arpaci-Dusseau, OSTEP

By the end of this lecture, you will be able to calculate turnaround and response times for any scheduling workload and explain the fundamental trade-offs between system throughput and interactivity.

| The Role of the Scheduler

In the previous lectures, we built the mechanical framework of CPU virtualization:

- Transitioning between User and Kernel modes.
- Regaining control of the CPU via timer interrupts.
- Swapping execution state via the context switch.

Now, we must build the **policy layer**:

- The mechanism lets us switch processes.
- The **scheduler** is the decision-maker that chooses *which* process gets the CPU.

Let's begin by setting up our analytical framework.

| Lecture Roadmap

This lecture is organized into **6 major parts**:

1. **Workload Assumptions & Metrics** — Setting up the rules of our scheduling sandbox
2. **FIFO and SJF** — Two basic policies and the convoy effect
3. **Preemption & STCF** — Preemption mechanics and optimal turnaround
4. **Round Robin (RR)** — Optimizing for response time and interactivity
5. **Incorporating I/O** — Real-world overlaps of CPU and I/O cycles
6. **Summary & Practical Problems** — Step-by-step problem-solving for exams

| Prerequisite Check

To follow this scheduling lecture, ensure you are comfortable with:

- **Process States:** Ready (waiting for CPU), Running (on CPU), Blocked (waiting for I/O).
- **Context Switch:** The hardware/software overhead of changing the running process.
- **Preemption:** The CPU being interrupted via timer ticks to force a mode switch.
- **Queues:** Basic queue operations (enqueue, dequeue).

Let's start by defining our scheduling workload.

§ 1 — Workload Assumptions & Metrics

| Defining the Workload

To design scheduling policies, we must model our processes (jobs) as a **workload**.

A workload describes:

- How many processes are running in the system.
- When each process arrives.
- How long each process needs the CPU to complete.
- Whether the processes perform input/output (I/O) operations.

To make progress, we start with a set of highly simplified assumptions about our workload.

| The Five Workload Assumptions

Initially, we assume:

1. **Equal Run-time:** Every job requires the exact same amount of time to execute.
2. **Simultaneous Arrival:** All jobs arrive at the system at the exact same time ($t = 0$).
3. **Run to Completion:** Once a job starts, it runs until it is completely finished (no preemption).
4. **CPU Only:** Jobs only use the CPU; they perform no I/O operations.
5. **Known Run-time:** The exact run-time of each job is known to the scheduler in advance.

| Assumption 1: Equal Run-time

What does this mean?

- Every process requires the same number of CPU cycles.
- For example, Jobs A, B, and C all take exactly 10 seconds of CPU execution.
- Real-world violation: A compiler might run for 3 minutes, while a cursor movement handler runs for 1 millisecond.

| Assumption 2: Simultaneous Arrival

What does this mean?

- All processes are submitted to the OS run queue at time $t = 0$.
- The scheduler has all choices available immediately.
- Real-world violation: Users start programs dynamically; processes spawn children at arbitrary points in time.

| **Assumption 3: Run to Completion**

What does this mean?

- Once the scheduler assigns the CPU to a job, that job retains the CPU until it completes.
- There are no timer interrupts forcing preemption.
- Real-world violation: Modern systems use preemptive context switching to interrupt long-running tasks.

| Assumption 4: CPU Only

What does this mean?

- The process never blocks.
- It executes arithmetic, logic, and memory operations continuously.
- Real-world violation: Almost all interactive processes perform network, disk, or keyboard I/O regularly, entering the blocked state.

| Assumption 5: Known Run-time

What does this mean?

- The OS is omniscient. It knows in advance that Job A will run for exactly 12.4 seconds.
- Real-world violation: The **Halting Problem*** proves that there is no **general algorithm** that can determine if/when an **arbitrary program** will eventually stop
- Therefore, the OS cannot, in general, know **whether a program will finish, let alone exactly when**.

* A. M. Turing, "On Computable Numbers, with an Application to the Entscheidungsproblem" (1936).

DOI: [10.1112/plms/s2-42.1.230](https://doi.org/10.1112/plms/s2-42.1.230)

| Why Start with Simplifications?

These assumptions are completely unrealistic:

- Jobs run for vastly different lengths of time.
- Jobs arrive at different times.
- Jobs perform extensive I/O (waiting for disk, network, user).
- We can rarely predict how long a program will run before it executes.

However, starting with these simplifications allows us to understand the core logic of each policy. We will relax each assumption one-by-one.

| Scheduling Metrics

To evaluate scheduling policies, we need objective, quantitative **metrics**.

We use two primary metrics in this lecture:

Scheduling Metrics	
Turnaround Time (Optimizes efficiency)	Response Time (Optimizes interactivity)

Let's define each metric mathematically.

Metric 1: Turnaround Time

Turnaround time is the total time that elapses from when a job arrives at the system to when it completes.

Formula:

$$T_{\text{turnaround}} = T_{\text{completion}} - T_{\text{arrival}}$$

- Since all jobs currently arrive at $t = 0$:

$$T_{\text{turnaround}} = T_{\text{completion}}$$

- We average this metric over all jobs to get the **Average Turnaround Time**.

Metric 2: Response Time

With the rise of interactive time-sharing systems, turnaround time is not enough. Users need quick feedback.

***Response time** is the time from when a job arrives to when it is scheduled for the first time.*

Formula:

$$T_{\text{response}} = T_{\text{firstrun}} - T_{\text{arrival}}$$

- If a job arrives at $t = 0$ but is not scheduled until $t = 5$, its response time is 5s.

| Fairness vs. Performance

We also consider **Fairness**:

- How evenly is CPU execution time shared among all processes?
- A scheduling policy can be highly efficient (low average turnaround) but unfair (some jobs wait a very long time).

Performance (minimizing turnaround/response) and Fairness are often in conflict. We will see how scheduling algorithms manage this trade-off.

| Quantifying Fairness: Jain's Index

To mathematically evaluate fairness, we use **Jain's Fairness Index**:

$$J(x_1, x_2, \dots, x_n) = \frac{(\sum_{i=1}^n x_i)^2}{n \cdot \sum_{i=1}^n x_i^2}$$

- Where x_i is the share of CPU time allocated to process i .
- If all n processes get an equal share, $J = 1$ (perfect fairness).
- If one process monopolizes the CPU, J approaches $1/n$ (worst fairness).

✓ Checkpoint 1 — Assumptions & Metrics

Test your understanding:

1. List the five workload assumptions we start with. Which one is the most unrealistic?
2. Write the formula for **Turnaround Time**. How does it differ from **Response Time**?
3. In a multi-user interactive system, why is response time often preferred over turnaround time as a primary metric?
4. Explain the conceptual conflict between scheduling efficiency and fairness.

Ensure you have these definitions clear before analyzing scheduling algorithms.

§ 2 — FIFO and SJF

| **First-In, First-Out (FIFO)**

The simplest scheduling policy is **First-In, First-Out** (FIFO), also known as **First-Come, First-Served** (FCFS).

- Jobs are scheduled in the order they arrive.
- When a job gets the CPU, it runs until completion.
- Simple to implement: uses a standard FIFO queue.

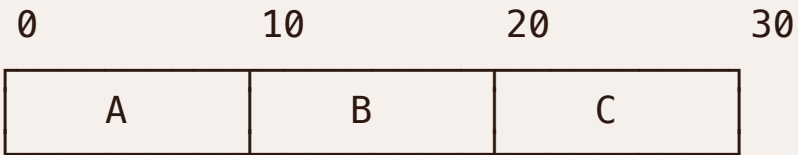
Let's evaluate FIFO under our five basic assumptions.

FIFO under Basic Assumptions

Suppose we have three jobs: **A**, **B**, and **C**.

- Arrival time: $t = 0$ for all jobs.
- Run-time: 10 seconds each (Assumption 1: equal run-times).

Timeline:



- **A** completes at $t = 10$. Turnaround = 10s.
- **B** completes at $t = 20$. Turnaround = 20s.
- **C** completes at $t = 30$. Turnaround = 30s.

Average Turnaround Time = $\frac{10+20+30}{3} = \mathbf{20}$ seconds.

| Relaxing Assumption 1

Now, let's relax **Assumption 1** (jobs no longer have equal run-times).

New workload:

- **A** runs for 100 seconds.
- **B** and **C** run for 10 seconds each.
- All arrive at $t = 0$.

Let's see what happens if the scheduler runs **A**, then **B**, then **C**.

The Convoy Effect Visualized

Timeline:



Let's calculate the turnaround times:

- **A** completes at $t = 100$. Turnaround = 100s.
- **B** completes at $t = 110$. Turnaround = 110s.
- **C** completes at $t = 120$. Turnaround = 120s.

Average Turnaround Time = $\frac{100+110+120}{3} = \mathbf{110}$ seconds.

*The short jobs B and C are stuck behind the long job A. This is the **Convoy Effect**.*

Understanding the Convoy Effect

The Convoy Effect occurs when several relatively short processes are queued behind a heavy, long-running process, resulting in poor average wait and turnaround times.

Analogy:

- A supermarket line where customers with one item are stuck behind a customer with a full cart.
- The cashier could minimize average waiting time by checking out the quick customers first.

This leads us to a new scheduling policy.

| Shortest Job First (SJF)

To resolve the convoy effect, we introduce **Shortest Job First** (SJF):

- When selecting the next job to run, the scheduler chooses the one with the shortest run-time.
- Non-preemptive: Once a job starts, it runs to completion.

Let's evaluate the same workload under SJF:

- **A** runs for 100s.
- **B** and **C** run for 10s each.
- All arrive at $t = 0$.

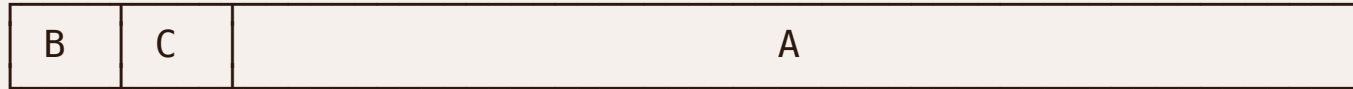
| SJF Workload Trace

Since B and C are shorter (10s) than A (100s), SJF schedules B and C first:

Timeline:

0 10 20

120



Turnaround calculations:

- **B** completes at $t = 10$. Turnaround = 10s.
- **C** completes at $t = 20$. Turnaround = 20s.
- **A** completes at $t = 120$. Turnaround = 120s.

Average Turnaround Time = $\frac{10+20+120}{3} = \mathbf{50}$ seconds.

By changing the execution order, we reduced the average turnaround from 110s to 50s!

| The Optimality of SJF

SJF is a highly significant algorithm because:

*SJF is **optimal** for minimizing the average turnaround time, provided that all jobs arrive at the same time.*

Proof concept:

- Running shorter jobs first moves their completion times earlier without delaying the remaining work.
- This mathematically guarantees the lowest possible average turnaround time for non-preemptive workloads arriving simultaneously.

| Proof: Minimizing Average Turnaround

Let's represent the turnaround time of N jobs as:

$$\sum_{i=1}^N T_{\text{turnaround},i} = \sum_{i=1}^N (N - i + 1) \cdot R_i$$

Where R_i is the run-time of the i -th scheduled job.

- The first job's run-time R_1 contributes to the completion time of all N jobs.
- The last job's run-time R_N only contributes to its own completion.
- To minimize the sum, we must assign the smallest values of R_i to the largest coefficients $(N - i + 1)$.
- Therefore, we must sort the run-times in ascending order: $R_1 \leq R_2 \leq \dots \leq R_N$.

| Relaxing Assumption 2

Now, let's relax **Assumption 2** (jobs can arrive at different times).

New workload:

- **A** runs for 100 seconds, but arrives at $t = 0$.
- **B** and **C** run for 10 seconds each, but arrive at $t = 10$.

Let's see what happens under non-preemptive SJF.

| SJF with Varying Arrivals Trace

At $t = 0$, only job **A** is in the queue. The scheduler must run **A**:

Timeline:

0	10 (B, C arrive)		100	110	120
A			B	C	

- **A** completes at $t = 100$. Turnaround = $100 - 0 = 100$ s.
- **B** completes at $t = 110$. Turnaround = $110 - 10 = 100$ s.
- **C** completes at $t = 120$. Turnaround = $120 - 10 = 110$ s.

Average Turnaround Time = $\frac{100+100+110}{3} = \mathbf{103.3}$ seconds.

Even though B and C arrived at $t = 10$, they had to wait for A to finish. The convoy effect is back.

✓ Checkpoint 2 — FIFO and SJF

Test your understanding:

1. What is the convoy effect? Describe a real-world scenario that illustrates it.
2. In what scenario is SJF mathematically optimal for average turnaround time?
3. Explain why the convoy effect can return in SJF if we relax the assumption of simultaneous arrival.
4. Draw the Gantt chart and calculate average turnaround for:
 - Jobs: X (runtime 30s), Y (10s), Z (5s).
 - All arrive at $t = 0$.
 - Policies: FIFO vs. SJF.

Make sure you can calculate these numbers easily before moving on.

§ 3 — Preemption & STCF

| Introducing Preemption

To resolve the convoy effect with varying arrival times, we must relax **Assumption 3** (jobs run to completion).

The scheduler needs the ability to stop a running process.

- **Preemptive Scheduler:** Can interrupt a running process, save its context, and switch to another process.
- **Non-preemptive Scheduler:** Once a process starts, it retains the CPU until it exits or blocks.

Preemption requires the hardware timer interrupt mechanism.

| Shortest Time-to-Complete First (STCF)

With preemption, we can improve SJF: **Shortest Time-to-Complete First (STCF)**, also known as **Preemptive Shortest Job First (PSJF)**.

- When a new job enters the system, the scheduler compares the remaining time of the running job with the run-time of the new job.
- If the new job has a shorter remaining run-time, the scheduler preempts the current job and starts the new job.

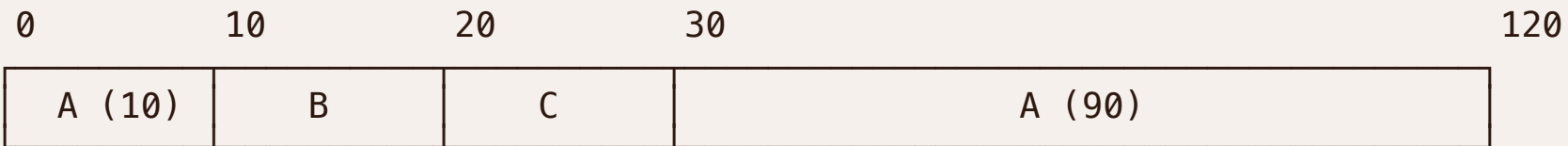
Let's evaluate our previous workload under STCF.

| STCF Workload Walkthrough

Workload:

- **A** runs for 100s, arrives at $t = 0$.
- **B** and **C** run for 10s each, arrive at $t = 10$.

Timeline:



1. At $t = 0$, only **A** runs.
2. At $t = 10$, **B** and **C** arrive.
 - Remaining time for **A**: 90s.
 - Run-times of **B** and **C**: 10s each.
 - **B** and **C** are shorter, so **A** is preempted. The scheduler runs **B**.

| Calculating STCF Metrics

Continuing our STCF trace:

- At $t = 10$, **B** runs and completes at $t = 20$.
- At $t = 20$, **C** runs and completes at $t = 30$.
- At $t = 30$, **A** resumes and completes at $t = 120$.

Turnaround calculations:

- **A** completes at $t = 120$. Turnaround = $120 - 0 = 120\text{s}$.
- **B** completes at $t = 20$. Turnaround = $20 - 10 = 10\text{s}$.
- **C** completes at $t = 30$. Turnaround = $30 - 10 = 20\text{s}$.

Average Turnaround Time = $\frac{120+10+20}{3} = \mathbf{50}$ seconds.

STCF reduced average turnaround from 103.3s to 50s!

| The Optimality of STCF

STCF is a key algorithm because:

*STCF is **optimal** for minimizing average turnaround time when jobs can arrive at different times.*

Proof concept:

- By always prioritizing the job that is closest to completion, STCF minimizes the queue size and ensures short jobs finish as quickly as possible.
- This mathematically guarantees the minimum average turnaround time for any preemptive workload.

| The Problem of Starvation

Although STCF is optimal for average turnaround, it has a severe drawback: **Starvation**.

- If short jobs arrive continuously, the scheduler will repeatedly preempt or postpone any long-running jobs.
- The long-running jobs may never complete, starving for CPU cycles.

```
Queue state over time:  
t=0:   Job A (100s) runs.  
t=10:  Job B (5s) arrives, preempts A.  
t=14:  Job C (5s) arrives, runs before A.  
t=18:  Job D (5s) arrives... (A never runs!)
```

This is an issue of **Fairness**.

| The Unrealistic Assumption 5

Both SJF and STCF rely on **Assumption 5**: the scheduler knows the exact run-time of each job in advance.

In real systems, this is impossible:

- A text editor might run for 5 seconds or 5 hours.
- A compiler's run-time depends on file size and compilation options.
- The OS cannot predict the future.

We will see in Lecture 2.3 how the Multi-Level Feedback Queue (MLFQ) uses past behavior to predict future run-times.

✓ Checkpoint 3 — Preemption & STCF

Test your understanding:

1. Explain the difference between preemptive and non-preemptive scheduling.
2. How does STCF decide whether to preempt the currently running process?
3. What is scheduling starvation? How can it occur under STCF?
4. If STCF is optimal for average turnaround time, why is it rarely implemented in its pure form in modern general-purpose operating systems?

Verify your understanding of these trade-offs before we look at response time optimization.

§ 4 — Round Robin (RR)

| The Need for Interactivity

Suppose we are running an interactive system:

- User A is editing a text file.
- User B is compiling a program.
- If both jobs arrive together, SJF/STCF will run the shorter one to completion before starting the other.

If User B's compilation runs first for 10 seconds:

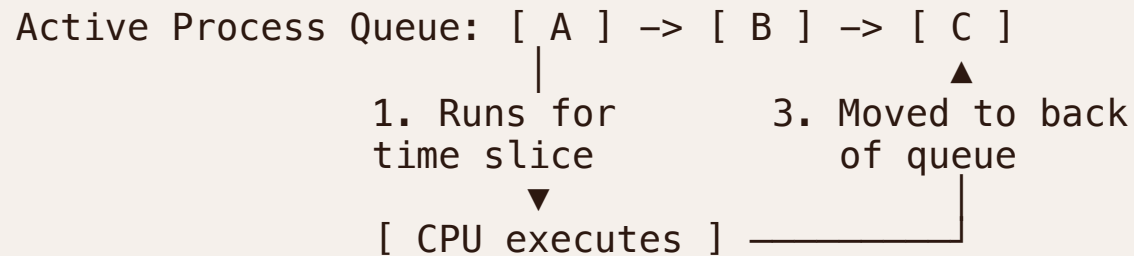
- User A's keystrokes are ignored for 10 seconds.
- Response time is terrible.
- Users expect immediate feedback (response time $< 100\text{ms}$).

To optimize response time, we need a new scheduling approach.

Round Robin (RR) Concept

Round Robin (RR), also known as **Time-slicing**, schedules jobs in a circular loop:

- Instead of running jobs to completion, RR runs a job for a short time slice.
- It then moves the job to the back of the queue and runs the next one.
- This process repeats until all jobs are finished.



| The Time Slice (Scheduling Quantum)

The length of the execution period is called the **Time Slice** or **Scheduling Quantum**.

- Typically a multiple of the hardware timer interrupt period (e.g., 10ms to 100ms).
- Must be long enough to amortize context switch overhead.
- Must be short enough to maintain the illusion of simultaneous execution for users.

The OS relies on the timer interrupt to force the process to yield when its time slice expires.

Round Robin Walkthrough

Workload:

- **A**, **B**, and **C** run for 10 seconds each, arriving at $t = 0$.
- Time slice: 2 seconds.

Timeline:

0	2	4	6	8	10	12	14	16	18	20	...	30
A	B	C	A	B	C	A	B	C	A	...		

The scheduler cycles through the jobs: A, B, C, A, B, C, and so on.

Let's calculate the response times.

| Response Time Comparison

Let's compare the response times of SJF vs. RR for this workload:

- **SJF (non-preemptive):**
 - **A** starts at $t = 0$. Response = 0s.
 - **B** starts at $t = 10$. Response = 10s.
 - **C** starts at $t = 20$. Response = 20s.
 - Average Response Time = $\frac{0+10+20}{3} = \mathbf{10}$ seconds.
- **Round Robin (2s slice):**
 - **A** starts at $t = 0$. Response = 0s.
 - **B** starts at $t = 2$. Response = 2s.
 - **C** starts at $t = 4$. Response = 4s.
 - Average Response Time = $\frac{0+2+4}{3} = \mathbf{2}$ seconds.

RR significantly reduces the average response time!

| The Time Slice Trade-off

The choice of time slice length is critical for system performance:

Time Slice Tuning Trade-offs	
Too Short (e.g., 1ms) - Excellent response time. - High context switch cost (wastes CPU cycles).	Too Long (e.g., 500ms) - Low context switch cost. - Poor response time. - Degenerates to FIFO.

The OS designer must choose a balance, typically keeping context switch overhead under 1% of total CPU time.

| Quantifying Context Switch Overhead

Suppose:

- Time slice is set to 10ms.
- Context switch time (saving registers, cache recovery) takes 1ms.

$$\text{Overhead} = \frac{\text{Switch Time}}{\text{Time Slice} + \text{Switch Time}} = \frac{1\text{ms}}{10\text{ms} + 1\text{ms}} \approx \mathbf{9\%}$$

9% of the CPU's processing power is wasted on OS overhead!

If we increase the time slice to 100ms:

$$\text{Overhead} = \frac{1\text{ms}}{100\text{ms} + 1\text{ms}} \approx \mathbf{0.99\%}$$

Now, less than 1% of CPU time is wasted on overhead.

| RR and Turnaround Time

While Round Robin is excellent for response time, it is terrible for turnaround time.

Let's look at the completion times for A, B, C under RR (10s run-time each, 2s time slice):

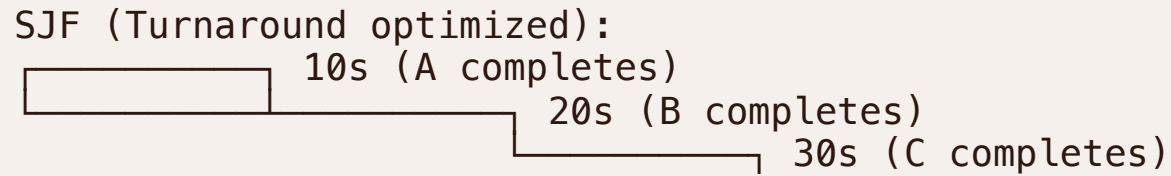
- **A** completes at $t = 26$. Turnaround = 26s.
- **B** completes at $t = 28$. Turnaround = 28s.
- **C** completes at $t = 30$. Turnaround = 30s.

Average Turnaround Time = $\frac{26+28+30}{3} = \mathbf{28}$ seconds.

Recall that under FIFO/SJF, the average turnaround was **20 seconds**.

| Why RR has Poor Turnaround Time

SJF runs jobs sequentially, completing each as fast as possible:



Round Robin stretches out the completion times of all jobs:



By sharing the CPU evenly, we delay the completion of every single job.

| Turnaround vs. Response Trade-off

This illustrates a fundamental trade-off in scheduler design:

*Any policy that is **fair** and optimizes **response time** (like Round Robin) will perform poorly for **turnaround time**.*

*Any policy that optimizes **turnaround time** (like SJF/STCF) will perform poorly for **response time**.*

You cannot optimize both simultaneously. The OS must choose based on the workload requirements.

✓ Checkpoint 4 — Round Robin

Test your understanding:

1. Explain how Round Robin scheduling improves response times for interactive processes.
2. Why is Round Robin poor for average turnaround time compared to SJF?
3. Calculate the context switch overhead percentage if the time slice is 5ms and the context switch cost is 0.5ms.
4. What happens to the behavior of Round Robin as the time slice approaches infinity? What happens as it approaches zero?

Be ready to discuss these trade-offs in detail.

§ 5 — Incorporating I/O Operations

| Relaxing Assumption 4

Real programs do not just run calculations on the CPU. They perform I/O operations:

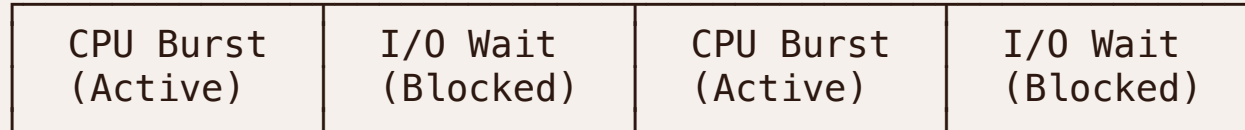
- A program reads input from a database on disk.
- A web server waits for a network socket connection.
- An editor waits for a key press.

When a process initiates an I/O request:

- It transitions to the **BLOCKED** state.
- It cannot use the CPU until the I/O operation completes.
- The scheduler should run another process in the meantime to keep the CPU busy.

| The Nature of I/O Bursts

A typical program alternates between CPU execution and I/O wait:



- **CPU Burst:** The active processing phase.
- **I/O Wait:** The blocked waiting phase.

The scheduler's goal is to overlap these phases across multiple processes to maximize resource utilization.

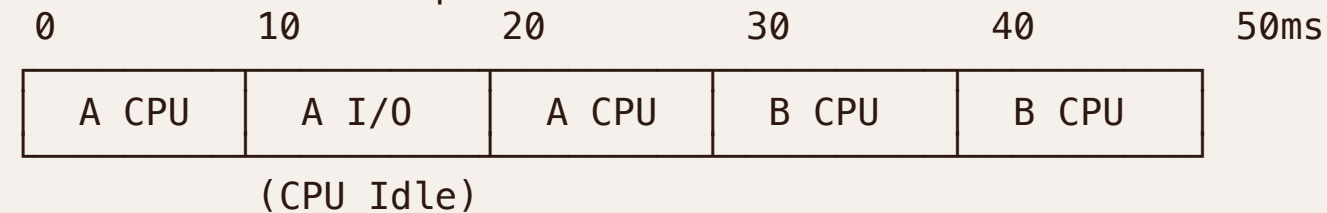
| Non-Overlapped I/O Trace

Suppose we run two jobs, **A** (CPU and I/O) and **B** (CPU only), without overlapping:

Job A: Runs CPU 10ms -> waits I/O 10ms -> runs CPU 10ms

Job B: Runs CPU 20ms

Timeline (no overlap):



The CPU sits idle during step 2 while A waits for the disk, wasting execution cycles.

Overlapped I/O Trace

By overlapping operations, we can keep the CPU busy while A is waiting for I/O:

Timeline (overlapped):

0 10 20 30 40ms



▲
A blocks on I/O;
OS schedules B on CPU

- When A blocks on I/O at $t = 10$, the OS switches the CPU to B.
- When A's I/O completes at $t = 20$, the OS makes A runnable again.
- This maximizes CPU utilization.

| Splitting into Sub-jobs

How do we represent this in our scheduling algorithms?

We treat each CPU burst of a process as an independent **sub-job**:

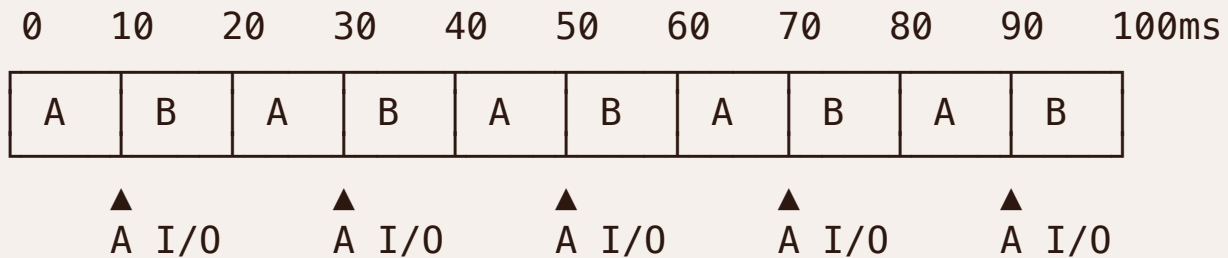
- If Job A needs 50ms of CPU total, split into five 10ms CPU bursts separated by I/O.
- The scheduler treats each burst as a short 10ms job.
- Under SJF/STCF, these short bursts are scheduled quickly, ensuring interactive response.

I/O Overlap with STCF

Let's see how STCF handles a mixed CPU-I/O workload:

- **A** runs for 10ms CPU, then waits 10ms I/O. (Repeat 5 times).
- **B** is a CPU-bound job that runs for 50ms.

Timeline:



- A is scheduled first because its burst (10ms) is shorter than B's total time (50ms).
- When A blocks, B runs.
- When A's I/O completes, A preempts B because A's remaining burst (10ms) is shorter than B's remaining time.

This keeps both the CPU and the I/O device busy.

✓ Checkpoint 5 — Incorporating I/O

Test your understanding:

1. What process state transition occurs when a process initiates a disk read operation?
2. Explain how overlapping CPU execution and I/O waiting improves system efficiency.
3. How can a scheduler like STCF handle I/O-bound jobs by splitting them into sub-jobs?
4. What is the main risk if a CPU-bound job is scheduled concurrently with many I/O-bound jobs under STCF?

Ensure you understand I/O overlap before moving to the final mathematical walkthroughs.

§ 6 — Summary & Practical Problems

| Comparison of Basic Policies

Here is a summary of our basic scheduling policies:

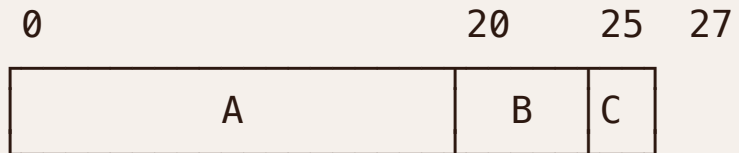
Policy	Decision Rule	Preemptive?	Optimizes?	Disadvantages
FIFO	Arrival order	No	Simplicity	Convoy Effect
SJF	Shortest run-time	No	Turnaround	Assumes run-time is known; Poor response
STCF	Shortest remaining time	Yes	Turnaround	Assumes run-time is known; Starvation
RR	Circular time slices	Yes	Response	Poor turnaround; Context switch overhead

| Exam Practice: Problem 1

Let's trace a simple workload.

- **Jobs:** A (20s runtime), B (5s), C (2s).
- **Arrivals:** All at $t = 0$.
- We calculate average turnaround and response times under FIFO (order A, B, C) and SJF.

Problem 1: FIFO Timeline



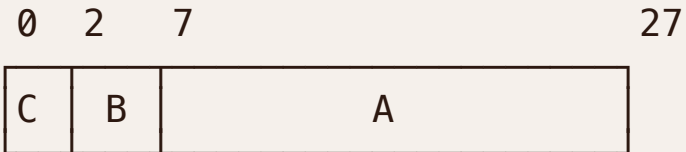
Turnaround Calculations ($T_c - T_a$):

- **A:** $20 - 0 = 20\text{s}$
- **B:** $25 - 0 = 25\text{s}$
- **C:** $27 - 0 = 27\text{s}$
- **Avg Turnaround:** $\frac{20+25+27}{3} = 24\text{s}$

Response Calculations ($T_f - T_a$):

- **A:** 0s, **B:** 20s, **C:** 25s
- **Avg Response:** $\frac{0+20+25}{3} = 15\text{s}$

Problem 1: SJF Timeline



Turnaround Calculations ($T_c - T_a$):

- **C:** $2 - 0 = 2\text{s}$
- **B:** $7 - 0 = 7\text{s}$
- **A:** $27 - 0 = 27\text{s}$
- **Avg Turnaround:** $\frac{2+7+27}{3} = \mathbf{12\text{s}}$

Response Calculations ($T_f - T_a$):

- **C:** 0s, **B:** 2s, **A:** 7s
- **Avg Response:** $\frac{0+2+7}{3} = \mathbf{3\text{s}}$

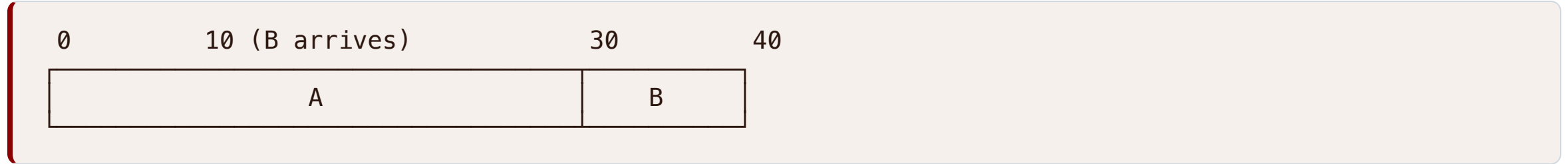
| Exam Practice: Problem 2

Let's run a workload with varying arrival times.

- **Jobs:** A (runtime 30s, arrival $t = 0$), B (runtime 10s, arrival $t = 10$).
- We compare **Non-preemptive SJF** and **Preemptive STCF**.

Problem 2: SJF (Non-preemptive) Trace

At $t = 0$, only A is in the queue. A runs to completion ($t = 30$):



Turnaround Calculations ($T_c - T_a$):

- **A:** $30 - 0 = 30\text{s}$
- **B:** $40 - 10 = 30\text{s}$
- **Avg Turnaround:** $\frac{30+30}{2} = 30\text{s}$

Response Calculations ($T_f - T_a$):

- **A:** $0 - 0 = 0\text{s}$
- **B:** $30 - 10 = 20\text{s}$
- **Avg Response:** $\frac{0+20}{2} = 10\text{s}$

Problem 2: STCF (Preemptive) Trace

At $t = 10$, B arrives.

- Remaining A = 20s and B = 10s ==> B is shorter, so B preempts A and runs to completion:

0	10	20	40
A (10)	B	A (20)	

Turnaround Calculations ($T_c - T_a$):

- B:** $20 - 10 = 10\text{s}$
- A:** $40 - 0 = 40\text{s}$
- Avg Turnaround:** $\frac{10+40}{2} = 25\text{s}$

Response Calculations ($T_f - T_a$):

- A:** $0 - 0 = 0\text{s}$, **B:** $10 - 10 = 0\text{s}$
- Avg Response:** 0s

| Exam Practice: Problem 3 (Comprehensive)

Workload:

- **Process A:** Arrival $t = 0$, Run-time = 40s
- **Process B:** Arrival $t = 10$, Run-time = 20s
- **Process C:** Arrival $t = 15$, Run-time = 10s

We will calculate the **Average Turnaround Time** and **Average Response Time** under:

1. **FIFO**
2. **SJF** (non-preemptive)
3. **STCF**
4. **Round Robin** (Time slice = 5s)

Policy 1: FIFO Solution

Under FIFO, jobs run in order of arrival: A, then B, then C.

Gantt Chart:



- **Arrivals:** A=0, B=10, C=15
- **Completion Times (T_c):** A=40, B=60, C=70
- **First Run Times (T_f):** A=0, B=40, C=60

| FIFO Math Calculations

Turnaround Times ($T_c - T_a$):

- **A:** $40 - 0 = 40\text{s}$
- **B:** $60 - 10 = 50\text{s}$
- **C:** $70 - 15 = 55\text{s}$
- **Average Turnaround Time** = $\frac{40+50+55}{3} = 48.33$ seconds

Response Times ($T_f - T_a$):

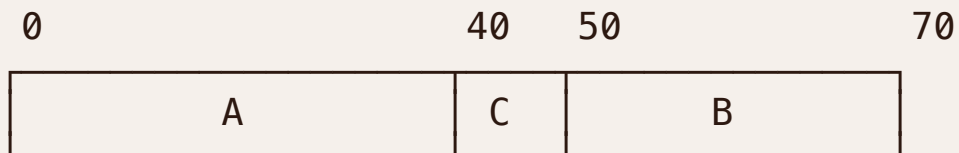
- **A:** $0 - 0 = 0\text{s}$
- **B:** $40 - 10 = 30\text{s}$
- **C:** $60 - 15 = 45\text{s}$
- **Average Response Time** = $\frac{0+30+45}{3} = 25$ seconds

| Policy 2: SJF (Non-preemptive) Solution

SJF is non-preemptive, so when a job starts, it runs to completion.

- At $t = 0$, only **A** is present. **A** starts running.
- During A's execution, **B** (at $t = 10$) and **C** (at $t = 15$) arrive in the queue.
- At $t = 40$, **A** completes. The scheduler compares the run-times of queue jobs:
 - **B** needs 20s.
 - **C** needs 10s.
- **C** is shorter, so it runs first, followed by **B**.

Gantt Chart:



| SJF Turnaround Calculations

- **Arrivals:** A=0, B=10, C=15
- **Completion Times (T_c):** A=40, C=50, B=70
- **First Run Times (T_f):** A=0, C=40, B=50

Turnaround Times ($T_c - T_a$):

- **A:** $40 - 0 = 40\text{s}$
- **B:** $70 - 10 = 60\text{s}$
- **C:** $50 - 15 = 35\text{s}$
- **Average Turnaround Time** = $\frac{40+60+35}{3} = 45$ seconds

| SJF Response Calculations

- **Arrivals:** A=0, B=10, C=15

Response Times ($T_f - T_a$):

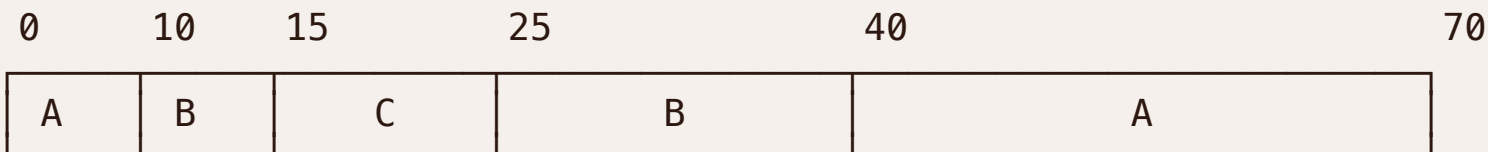
- **A:** $0 - 0 = 0\text{s}$
- **B:** $50 - 10 = 40\text{s}$
- **C:** $40 - 15 = 25\text{s}$
- **Average Response Time** = $\frac{0+40+25}{3} = \mathbf{21.67}$ seconds

Policy 3: STCF (Preemptive) Solution

STCF compares remaining run-times when new jobs arrive:

- At $t = 0$, **A** starts running.
- At $t = 10$, **B** (20s runtime) arrives.
 - Remaining time for **A**: 30s.
 - **B** is shorter (20s vs 30s). **A** is preempted; **B** starts running.
- At $t = 15$, **C** (10s runtime) arrives.
 - Remaining times: **B**=15s, **A**=30s.
 - **C** is shorter (10s vs 15s). **B** is preempted; **C** starts running.
- At $t = 25$, **C** completes. Remaining times: **B**=15s, **A**=30s. **B** resumes.
- At $t = 40$, **B** completes. **A** resumes and runs to completion at $t = 70$.

Gantt Chart:



| STCF Turnaround Calculations

- **Arrivals:** $A=0$, $B=10$, $C=15$
- **Completion Times (T_c):** $C=25$, $B=40$, $A=70$
- **First Run Times (T_f):** $A=0$, $B=10$, $C=15$

Turnaround Times ($T_c - T_a$):

- **A:** $70 - 0 = 70\text{s}$
- **B:** $40 - 10 = 30\text{s}$
- **C:** $25 - 15 = 10\text{s}$
- **Average Turnaround Time** = $\frac{70+30+10}{3} = \mathbf{36.67}$ seconds

| STCF Response Calculations

- **Arrivals:** A=0, B=10, C=15

Response Times ($T_f - T_a$):

- **A:** $0 - 0 = 0\text{s}$
- **B:** $10 - 10 = 0\text{s}$
- **C:** $15 - 15 = 0\text{s}$
- **Average Response Time** = $\frac{0+0+0}{3} = 0$ seconds

| Policy 4: Round Robin (Time Slice = 5s) Solution

We track the active job queue step-by-step:

- $t = 0$: **A** arrives. Queue: [A]. **A** runs for 5s.
- $t = 5$: **A** yields. Queue: [A]. **A** runs for 5s.
- $t = 10$: **B** arrives. Queue: [A, B] (**A** yields, moves behind **B**). **B** runs for 5s.
- $t = 15$: **C** arrives. Queue: [A, C, B] (**B** yields, moves behind **C**). **A** runs for 5s.
- $t = 20$: Queue: [C, B, A]. **C** runs for 5s.
- $t = 25$: Queue: [B, A, C]. **B** runs for 5s.
- $t = 30$: Queue: [A, C, B]. **A** runs for 5s.
- $t = 35$: Queue: [C, B, A]. **C** runs for 5s (completes!). Queue: [B, A].
- $t = 40$: Queue: [A, B]. **B** runs for 5s.
- $t = 45$: Queue: [A, B]. **A** runs for 5s.
- $t = 50$: Queue: [B, A]. **B** runs for 5s (completes!). Queue: [A].
- $t = 55$: **A** runs to completion.

Round Robin Gantt Chart

Let's draw the Gantt chart for this Round Robin execution sequence:

Gantt Chart (Slice = 5s):

0 5 10 15 20 25 30 35 40 45 50 55 70

A	A	B	A	C	B	A	C	B	A	B	A
---	---	---	---	---	---	---	---	---	---	---	---

- **Arrivals:** A=0, B=10, C=15
- **Completion Times (T_c):** C=40, B=55, A=70
- **First Run Times (T_f):** A=0, B=10, C=20

| Round Robin Math Calculations

Turnaround Times ($T_c - T_a$):

- **A:** $70 - 0 = 70\text{s}$
- **B:** $55 - 10 = 45\text{s}$
- **C:** $40 - 15 = 25\text{s}$
- **Average Turnaround Time** = $\frac{70+45+25}{3} = \mathbf{46.67}$ seconds

Response Times ($T_f - T_a$):

- **A:** $0 - 0 = 0\text{s}$
- **B:** $10 - 10 = 0\text{s}$
- **C:** $20 - 15 = 5\text{s}$
- **Average Response Time** = $\frac{0+0+5}{3} = \mathbf{1.67}$ seconds

| Summary of Results

Let's compare the results across all policies for our workload:

Metric	FIFO	SJF	STCF	RR (5s)
A Turnaround	40s	40s	70s	70s
B Turnaround	50s	60s	30s	45s
C Turnaround	55s	35s	10s	25s
Avg Turnaround	48.33s	45.00s	36.67s	46.67s
Avg Response	25.00s	21.67s	0.00s	1.67s

- **STCF** achieved the optimal Average Turnaround Time (36.67s).
- **RR** achieved a low Average Response Time (1.67s), demonstrating the trade-off.

| Exam Practice: Problem 4

Let's trace a slightly modified workload.

- **Process A:** Arrival $t = 0$, Run-time = 15s
- **Process B:** Arrival $t = 5$, Run-time = 30s
- **Process C:** Arrival $t = 10$, Run-time = 5s

Calculate average turnaround and response times under **STCF**.

Problem 4: STCF Execution Trace

- $t = 0$: **A** starts running (A needs 15s).
- $t = 5$: **B** (30s) arrives. Remaining A = 10s. A is shorter than B, so **A** continues.
- $t = 10$: **C** (5s) arrives. Remaining A = 5s. Remaining B = 30s. Remaining A and C are tied (5s).
 - Let's assume the scheduler continues the running job (**A**). **A** continues.
- $t = 15$: **A** completes. Queue: B (30s), C (5s). **C** runs.
- $t = 20$: **C** completes. **B** runs to completion.
- $t = 50$: **B** completes.

Gantt Chart:



| Problem 4: STCF Calculations

- **Arrivals:** $A=0$, $B=5$, $C=10$
- **Completion Times (T_c):** $A=15$, $C=20$, $B=50$; **First Run Times (T_f):** $A=0$, $C=15$, $B=20$

Turnaround Times ($T_c - T_a$):

- **A:** $15 - 0 = 15\text{s}$
- **B:** $50 - 5 = 45\text{s}$
- **C:** $20 - 10 = 10\text{s}$
- **Average Turnaround Time** $= \frac{15+45+10}{3} = \mathbf{23.33}$ seconds

Response Times ($T_f - T_a$):

- **A:** $0 - 0 = 0\text{s}$
- **B:** $20 - 5 = 15\text{s}$
- **C:** $15 - 10 = 5\text{s}$
- **Average Response Time** $= \frac{0+15+5}{3} = \mathbf{6.67}$ seconds

| Exam Practice: Problem 5

Let's test **Round Robin** with variable time slices on this workload:

- **Jobs:** A (runtime 10s), B (runtime 10s)
- **Arrivals:** both at $t = 0$.
- We calculate average response and turnaround under time slice = 2s and time slice = 10s.

| Problem 5: Time Slice = 2s

Execution order: A, B, A, B, A, B, A, B, A, B.

- **First runs:** A starts at 0s, B starts at 2s.
- **Completions:** A completes at 18s, B completes at 20s.

Turnaround:

- **A:** $18 - 0 = 18\text{s}$
- **B:** $20 - 0 = 20\text{s}$
- **Avg Turnaround:** 19s

Response:

- **A:** $0 - 0 = 0\text{s}$
- **B:** $2 - 0 = 2\text{s}$
- **Avg Response:** 1s

| Problem 5: Time Slice = 10s

This degenerates into FIFO because the time slice is equal to the run-times:

- **First runs:** A starts at 0s, B starts at 10s.
- **Completions:** A completes at 10s, B completes at 20s.

Turnaround:

- **A:** $10 - 0 = 10\text{s}$
- **B:** $20 - 0 = 20\text{s}$
- **Avg Turnaround:** 15s

Response:

- **A:** $0 - 0 = 0\text{s}$
- **B:** $10 - 0 = 10\text{s}$
- **Avg Response:** 5s

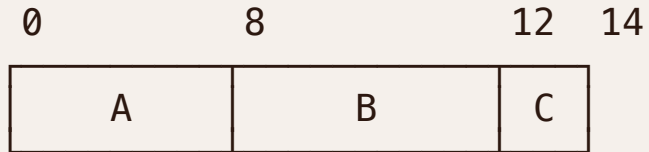
| Exam Practice: Problem 6

Trace this workload:

- **Jobs:** A (runtime 8s, arrival $t = 0$), B (runtime 4s, arrival $t = 2$), C (runtime 2s, arrival $t = 4$).
- Calculate metrics for **FIFO** and **STCF**.

Problem 6: FIFO Timeline & Math

FIFO execution: A [0-8], B [8-12], C [12-14].



Turnaround Calculations ($T_c - T_a$):

- **A:** $8 - 0 = 8s$
- **B:** $12 - 2 = 10s$
- **C:** $14 - 4 = 10s$
- **Avg Turnaround:** $\frac{8+10+10}{3} = \mathbf{9.33s}$

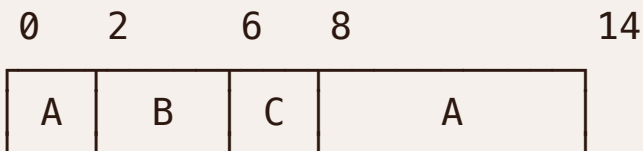
Response Calculations ($T_f - T_a$):

- **A:** $0s$, **B:** $8 - 2 = 6s$, **C:** $12 - 4 = 8s$
- **Avg Response:** $\frac{0+6+8}{3} = \mathbf{4.67s}$

Problem 6: STCF Timeline & Math

STCF execution:

- $t = 0$: A runs (A needs 8s).
- $t = 2$: B (4s) arrives. Remaining A = 6s. B is shorter, so **B** preempts A. B runs.
- $t = 4$: C (2s) arrives. Remaining B = 2s. Remaining A = 6s. B and C are tied (2s).
 - Let's assume the scheduler continues the running job (**B**). B continues.
- $t = 6$: B completes. Queue: C (2s), A (6s). **C** runs.
- $t = 8$: C completes. **A** resumes and runs to completion at $t = 14$.



| Problem 6: STCF Turnaround Calculations

- **Arrivals:** $A=0$, $B=2$, $C=4$
- **Completion Times (T_c):** $B=6$, $C=8$, $A=14$
- **First Run Times (T_f):** $A=0$, $B=2$, $C=6$

Turnaround Times ($T_c - T_a$):

- **A:** $14 - 0 = 14s$
- **B:** $6 - 2 = 4s$
- **C:** $8 - 4 = 4s$
- **Average Turnaround Time** = $\frac{14+4+4}{3} = \mathbf{7.33}$ seconds

| Problem 6: STCF Response Calculations

- **Arrivals:** A=0, B=2, C=4

Response Times ($T_f - T_a$):

- **A:** $0 - 0 = 0\text{s}$
- **B:** $2 - 2 = 0\text{s}$
- **C:** $6 - 4 = 2\text{s}$
- **Average Response Time** = $\frac{0+0+2}{3} = \mathbf{0.67}$ seconds

| Implementing Scheduler Run Queues in C

How is the scheduler's queue implemented in operating system code?

- **FIFO Queue:** A simple singly linked list. Enqueue at the tail is $O(1)$; Dequeue at the head is $O(1)$.
- **SJF/STCF (Sorted Queue):** An ordered linked list or binary heap.
 - Enqueue requires searching for the correct sorted location: $O(N)$ or $O(\log N)$.
 - Dequeue the shortest job is $O(1)$.
- **Round Robin:** A circular linked list. Dequeue head is $O(1)$, and Enqueue at tail is $O(1)$.

| Historical Context: The Evolution of Schedulers

CPU scheduling has evolved alongside computer hardware:

1. **Batch Systems (1950s-1960s):** FIFO and SJF. Computer time was expensive; throughput and turnaround were the only goals. Jobs ran to completion.
2. **Time-sharing Systems (1970s-Present):** Preemptive scheduling and Round Robin. Multiple interactive users sharing terminal systems, demanding low response times.
3. **Modern Multi-core Systems:** Complex load balancing, core affinities, and energy-aware scheduling.

| Exam Prep: More Practice Questions (1/2)

Try to answer these conceptual questions.

? Question 1

Under what workload conditions do FIFO and SJF produce identical scheduling timelines?

? Question 2

A scheduler uses a very small time slice (e.g., 0.1ms). What is the positive effect of this choice, and what is the negative effect?

| Exam Prep: More Practice Questions (2/2)

? Question 3

Why is STCF optimal for turnaround time, and why is it rarely used in desktop operating systems?



Answer Key & Explanations

Answer 1:

FIFO and SJF produce identical timelines when:

- All jobs arrive at the same time.
- All jobs require the exact same amount of execution time (or are sorted by length).

In this scenario, selecting by shortest length or arrival order results in the same sequence.

Answer 2:

- **Positive Effect:** Response time is extremely low. The system feels highly responsive to interactive inputs.
- **Negative Effect:** High context switch overhead. The CPU spends a significant fraction of its time saving/restoring register state rather than executing user code.

💡 Answer Key & Explanations (Cont.)

Answer 3:

- **Why Optimal:** By always completing the shortest remaining job first, STCF keeps the average wait time in the queue as low as possible.
- **Why Not Used in Desktop OS:**
 - i. It requires knowing the future run-time of processes, which the OS cannot predict.
 - ii. It can lead to starvation for long-running background tasks if the user continuously starts short tasks.

| Lecture 2.2 Complete

Recommended Reading

- OSTEP: Chapter 7
- Practice calculating turnaround times for various workloads

Contact & Q&A

- **Lecturer:** Gabriele Tolomei
- **Email:** tolomei@di.uniroma1.it
- **Office Hours:** Drop me a message, and we'll arrange a time to meet, either in person or online! 😊

Thank you for your attention!